

PRISM: 面向约束满足问题的范式引导推理与迭代求解器记忆

Anonymous Authors

摘要

本文提出 PRISM (Paradigm-guided Reasoning with Iterative Solver Memory), 一种面向约束满足问题 (CSP) 的记忆增强神经符号推理框架。现有神经符号方法通常将 LLM 与 Z3 等求解器结合, 但仍面临两类结构性局限: 修复反馈粒度较粗, 容易导致迭代停滞; 不同问题实例相互独立求解, 难以积累和复用跨实例经验。PRISM 的基本观点是: 在推理步骤可被形式化验证的任务中, 求解器可以作为经验准入的质量门控机制, 从而将 LLM 记忆由非结构化的自然语言启发式转化为可机械筛选的结构化推理经验。

为实现这一目标, PRISM 设计了两个相互补充的机制。第一, **SMT 筛选范式库** 将成功轨迹中的正向范式和失败轨迹中的错误范式统一编码为带触发条件、推理操作和求解器检查接口的结构化经验; 其中, 正向范式刻画可复用的推理操作, 错误范式则将 LLM 的系统性翻译失败模式 (如方向翻转、过度对称化、间距误用等) 编码为携带 UNSAT 签名的规避模板。第二, **类型化修复轨迹记忆** 基于 UNSAT core 归因记录错误类型、冲突约束和修复动作, 并通过停滞检测与代价分级策略切换缓解修复过程中的循环和停滞。

在 ZebraLogic (3×5 至 6×6 , 结构化逻辑网格) 与 AR-LSAT (231 道 LSAT 分析推理题, 包含排序、分组和分配类 CSP) 两类基准上的实验表明, PRISM 相比无记忆神经符号流程、自然语言经验记忆和直接检索式少样本基线均取得稳定提升; 在困难实例上, 其相对优势更为明显。实验结果表明, 将求解器验证前移到经验准入和修复归因阶段, 有助于比未验证的自然语言记忆更可靠地积累跨实例推理经验。

关键词: 约束满足问题; 神经符号推理; 智能体记忆; UNSAT core; 大语言模型; Z3

1 引言

约束满足问题 (Constraint Satisfaction Problem, CSP) 是一类要求系统在有限变量域上寻找满足全部约束赋值的推理任务, 其典型形式为在变量、值域和约束集合之间寻找全局一致解 [1]。CSP 广泛存在于任务规划、资源分配、调度优化、程序验证和逻辑推理等领域。

辑谜题求解等场景中。以逻辑网格谜题为代表的自然语言 CSP 进一步增加了问题难度：系统不仅需要从文本中识别变量、属性和关系，还需要在推理过程中维护跨约束的一致性，并避免局部正确推断导致全局冲突。

近年来，大语言模型（Large Language Model, LLM）在自然语言理解、数学推理和代码生成中表现出较强的泛化能力。然而，结构化约束推理暴露出不同于一般自然语言推理的难点。ZebraLogic 对逻辑网格谜题的系统评测表明，随着谜题规模增大和 Z3 冲突数上升，当前模型的求解精度会显著下降；增加 Best-of-N 采样、自我验证提示或更长推理链只能带来有限收益 [23]。AR-LSAT 对 LSAT 分析推理题的评测揭示了类似规律：纯链式思维推理（GPT-4 CoT）仅达 35%，只略高于五选一随机基线（20%）；Logic-LM 将 Z3 引入后提升至 43%，Logic-LM++ 进一步达到 46.3%，但整体仍远低于人类水平（ $\sim 90\%$ ）[7, 10]。这些结果共同表明：CSP 中的失败并非单纯源于推理预算不足，也与模型缺乏稳定、可复用且可验证的约束求解策略有关；这一瓶颈在不同类型的 CSP 任务中均有所体现。

神经符号推理为上述问题提供了重要路径。Logic-LM、LINC 和 SAT-LM 等工作将 LLM 的语言理解能力与 Z3、Prover9 等形式化求解器的确定性验证能力结合起来，形成“翻译、执行、修复”的基本流程 [7–9]。该范式中，LLM 负责将自然语言约束转化为符号表达式，求解器负责检查可满足性或执行推理，当求解失败时，LLM 再根据报错信息尝试修复翻译。相较于纯链式思维推理，这类方法能够利用求解器过滤部分不一致输出，因而在逻辑推理任务中取得更高可靠性。

尽管如此，现有神经符号系统仍存在两个主要限制。第一，修复反馈粒度不足。多数方法直接将求解器原始报错字符串或笼统的不满足性声明反馈给 LLM，难以指出矛盾具体来自哪些约束，也难以区分“当前新增推断错误”和“已有翻译错误”两类情形。第二，经验难以积累。现有流程通常将每道题视为独立实例，成功推理步骤、常见翻译错误和有效修复动作在任务结束后即被丢弃，导致系统在后续题目中重复出现同类错误。对于约束结构复杂、推理链较长的 CSP 实例，这两个问题会共同导致修复迭代停滞：模型反复修改相同约束或生成语义相近的修复动作，却无法改变 UNSAT 的主要来源。

智能体记忆研究从另一个角度探索了跨任务经验复用。Reflexion、ExpeL 和 ReasoningBank 等方法通过总结成功或失败轨迹，将自然语言反思、经验规则或结构化知识单元注入后续任务，从而提升智能体在网页导航、具身任务和软件工程等场景中的表现 [15, 16, 18]。然而，这类记忆机制直接用于 CSP 推理时存在粒度和可靠性错配。一方面，现有记忆多为任务级自然语言经验，而 CSP 求解需要的是推理操作级经验，例如当直接赋值约束与同属性互斥约束同时出现时，应如何传播域缩减。另一方面，自然语言记忆缺少形式语义，一条错误策略与一条正确策略在写入记忆库时缺少机械化区分标准，质量控制往往依赖 LLM 自评。

本文的核心观察是：CSP 场景具有通用智能体任务通常不具备的一项关键性质，即推理步骤可以由形式化求解器进行机械验证。基于这一性质，本文主张：**在推理步骤**

可被形式化验证的任务中，求解器可以作为经验准入的质量门控机制，从而使 LLM 记忆从自然语言启发式转变为可机械筛选的结构化推理经验。该主张在 PRISM 中落实为两类互补机制：

SMT 筛选范式库：将成功轨迹中的关键推理步骤提炼为正向范式，并将失败轨迹中的系统性翻译错误提炼为错误范式。正向范式对应 CSP 经典传播操作（直接赋值传播、链式位置推导等），但通过采样式 Z3 检查进行经验准入，无需手工编码领域理论。

类型化修复轨迹记忆：将失败修复轨迹中的 UNSAT 签名、错误类型和修复动作记录为结构化经验。实验发现，LLM 在 CSP 形式化中存在系统性且可重复的翻译失败模式，例如将有向邻接约束（“A 在 B 直接左侧”）对称化为双向 Or 表达式、将有向偏移方向（+1 vs -1）颠倒，或将邻接关系误编码为间距 +2 关系。这些错误模式在不同题目之间表现出较高一致性，难以在系统设计阶段事先穷举，但可以在运行时通过 UNSAT core 指纹自动识别并建立规避模板。

UNSAT core 指出导致矛盾的约束子集，为错误定位、修复目标选择和停滞检测提供了比原始报错更精确的结构化信号。

基于上述观察，本文提出 PRISM，一个面向 CSP 求解的记忆增强神经符号推理框架。PRISM 采用离线和在线两阶段架构。离线阶段从训练谜题的成功求解轨迹中提取关键决策点（Key Decision Point, KDP），对结构相似的推理步骤进行聚类，并借助 LLM 抽象为可复用的求解范式候选。每个候选范式包含触发条件、推断操作、Z3 前条件和 Z3 后条件，并在写入范式库前接受一致性、效果和触发精度三重筛选。在线阶段，系统根据当前约束状态检索高置信度范式并执行一致性预检，将通过检查的范式作为结构化提示注入 LLM；当 Z3 返回 UNSAT 时，系统提取 UNSAT core 进行归因，记录错误类型、冲突约束、修复动作和修复结果，形成类型化修复轨迹记忆。该记忆进一步通过 Jaccard 相似度检测 UNSAT core 停滞，通过动作相似度检测重复修复，并在必要时触发从切换修复目标到全量重新翻译的四级策略切换。

本文以 CSP 为例，提出以下具体贡献：

1. **SMT 筛选范式库：**区分正向范式（从成功轨迹提炼推理操作）与错误范式（从失败轨迹挖掘 UNSAT 签名模式），并以 Z3 的一致性、效果和触发精度检查作为经验准入信号。
2. **类型化修复轨迹记忆：**基于 UNSAT core 归因进行约束级错误定位，记录错误类型、冲突约束和修复动作，并通过 Jaccard 停滞检测和动作循环检测驱动四级策略切换。
3. **系统性实证验证：**在复杂 CSP 基准上比较无记忆神经符号流程、自然语言经验记忆、直接检索式少样本基线和 PRISM 消融变体，检验可机械筛选的结构化经验是否比未验证记忆更可靠。

2 相关工作

2.1 神经符号推理与约束满足求解

将大语言模型与形式化求解器结合是近年来逻辑推理研究中的重要方向。Logic-LM 提出由 LLM 完成符号翻译、由外部求解器执行推理、再由 LLM 解释或修复结果的三阶段框架 [7]。LINC 将自然语言问题转化为一阶逻辑并调用 Prover9 求解 [8]。SAT-LM 将自然语言问题形式化为 SAT 实例，总结了变量命名不一致、子句缺失等典型翻译错误 [9]。

后续工作进一步改进了翻译修复和约束定位能力。Logic-LM++ 利用候选修复方案的两两比较机制选择更优修复 [10]；VERUS-LM 关注与 IDP-Z3 知识库系统的可验证结合 [11]；Logic.py 通过面向逻辑谜题的领域特定语言降低翻译复杂度 [12]；VERGE 通过最小修正子集（MCS）定位冲突声明，并结合语义路由处理不同类型声明 [13]。

与 VERGE 的技术区别。 VERGE 的核心机制（MCS 定位）与本文的 UNSAT core 归因在使用的数学对象上存在差异：MCS 枚举最少需要移除哪些约束以恢复可满足性，计算代价较高；PRISM 使用 Z3 原生 UNSAT core（不可满足子集），代价较低，并且可直接用于约束级归因。进一步地，VERGE 将这一机制用于单题内的修复定位，而 PRISM 在此基础上建立跨题记忆库和四级策略切换机制；这两项组件并不属于 VERGE 的系统设计。

上述工作提高了单题求解能力，但普遍将每道题视为独立实例，不维护跨题推理经验，也不建模修复是否陷入停滞。CSP 基准研究揭示了这一缺口的重要性：ZebraLogic 以 Z3 冲突数衡量难度，指出当前 LLM 在复杂约束结构下存在显著的规模化失败 [23]；AR-LSAT 作为高难度 LSAT 分析推理基准，纯 CoT 推理仅达 35%（接近随机基线），加入符号求解器后 Logic-LM 提升至 43%，但仍远低于人类水平 [7, 24]；GridPuzzle 的细粒度错误分析表明，错误推理步骤和不当消去是逻辑网格谜题中的主要失败来源 [25]。这两类不同结构的 CSP 任务共同说明：LLM 在约束翻译中的系统性失败模式具有跨任务普遍性，而不是逻辑网格谜题所特有的现象。

2.2 大语言模型的迭代自修复

迭代自修复通过生成、反馈和修正循环提高 LLM 输出质量。Self-Refine 让模型对自身输出进行批评并修正 [14]；Reflexion 将任务失败后的自然语言反思写入情节记忆，用于后续尝试 [15]。此类方法取得积极结果，但也暴露出自反馈信号不稳定的问题：在缺少外部监督的情况下，LLM 往往不能可靠识别并修正自身推理错误。

在神经符号推理中，这一局限更加突出。若求解器只提供原始报错字符串，LLM 难以判断应修改哪条约束；修复多轮后准确率趋于平台，本质上反映了反馈信号信息量不足。本文使用 UNSAT core 替代原始报错作为主要修复信号，并通过类型化修复记录区分语法错误、过度约束、语义翻转和作用域错误等情形。

2.3 智能体记忆与经验蒸馏

智能体记忆方法尝试让 LLM 通过历史任务积累可复用经验。Reflexion 存储任务失败后的语言反思 [15]，ExpeL 通过成功与失败轨迹对比维护经验条目列表 [16]，AutoGuide 生成上下文感知的行动准则 [17]，ReasoningBank 将轨迹蒸馏为结构化知识单元并在测试时扩展推理 [18]。这些工作说明经验蒸馏能改善跨任务决策，但其记忆单元多数为自然语言文本，质量控制依赖 LLM 自评或任务级回报。

CSP 推理对记忆提出了更强要求：记忆需作用于推理操作级别而非任务级概述，且需要具备可被求解器检查的接口，否则错误经验会被反复注入。PRISM 的核心差异在于，它将记忆从自然语言经验转化为可机械筛选的结构化推理经验，只有通过一致性、效果和触发精度筛选的范式才能写入库中。

2.4 可复用推理规则学习

PRISM 与形式化方法中的推理规则学习传统密切相关。解释基学习（EBL）从成功轨迹中泛化出可复用模式 [19, 20]，能提供可证明的正确性，但依赖手工构建的领域理论。宏操作学习将原语动作序列压缩为可复用宏 [21]，Minton 分析了其覆盖率与匹配代价之间的效用权衡 [22]。在形式化求解器领域，冲突驱动子句学习（CDCL）[5] 和 SMT/模型检验中的引理生成 [6] 从失败分支积累经验反馈为新约束；最小不可满足子集枚举 [4] 是 PRISM 的 UNSAT core 分析的理论基础。

PRISM 在此谱系中占据特定位置：与 EBL 宏操作不同，PRISM 的范式从噪声较大的 LLM 轨迹中归纳得出，依靠采样式 SMT 筛选和在线一致性预检降低错误经验污染风险，而非依赖完备的手工领域理论；与 CDCL 引理学习不同，PRISM 的范式作用于跨问题实例的复用，而非单次求解器调用内的冲突子句；与 VERGE 等并发工作不同，PRISM 将 UNSAT core 分析用于跨题经验积累与停滞检测，而非限于单题修复定位。

3 问题定义与总体框架

3.1 问题定义与符号体系

本文研究的约束满足问题形式化为三元组

$$\mathcal{P} = (\mathbf{X}, \mathbf{D}, \mathbf{C}), \quad (1)$$

其中 $\mathbf{X} = \{x_1, x_2, \dots, x_n\}$ 是变量集合， $\mathbf{D} = \{D_1, D_2, \dots, D_n\}$ 是各变量的有限值域， $\mathbf{C} = \{c_1, c_2, \dots, c_m\}$ 是约束集合。一个解是完全赋值 $\theta : \mathbf{X} \rightarrow \bigcup_i D_i$ ，满足 $\theta(x_i) \in D_i$ 且对任意 $c_j \in \mathbf{C}$ 均有 $c_j(\theta)$ 成立。本文以逻辑网格谜题作为主要实例，变量对应实体与属性之间的关系，约束由自然语言给出，包含直接赋值、相邻关系、相对位置、互斥关系和绑定关系等类型。

表 1: 本文主要符号。

符号	含义
$\mathcal{P}$	CSP 问题实例
$\mathcal{T}$	求解轨迹
$S_t = (C_t, \delta_t)$	第 t 步求解器状态
a_t	第 t 步推理动作
U_t	第 t 步的 UNSAT core
P	求解范式
$\mathcal{L}$	范式库
$\mathcal{M}$	修复轨迹记忆
$\sigma(S_t)$	当前状态的约束类型签名
$\tau_{\text{snd}}, \tau_p$	范式筛选阈值（一致性、触发精度）
$\tau_{\text{stag}}, \tau_{\text{loop}}$	停滞和循环检测阈值

对于问题实例 $\mathcal{P}$ ，求解轨迹定义为状态与动作的序列：

$$\mathcal{T} = \langle (S_0, a_1, S_1), (S_1, a_2, S_2), \dots, (S_{T-1}, a_T, S_T) \rangle. \quad (2)$$

其中 $S_t = (C_t, \delta_t)$ 表示第 t 步求解器状态， $C_t \subseteq \mathbf{C}$ 为当前形式化约束集合， $\delta_t : \mathbf{X} \rightarrow 2^{\cup_i D_i}$ 为变量域状态； a_t 是 LLM 生成的推理动作，包含自然语言推断及其 Z3 形式化断言。若终止状态 S_T 中 Z3 返回 SAT 且赋值与标准答案一致，则轨迹为成功轨迹。表 1 汇总了本文主要符号。

3.2 PRISM 总体框架

PRISM 的总体思想是将经验积累与单题推理解耦。离线阶段负责从历史成功轨迹中提炼可机械筛选的范式，在线阶段负责将范式用于当前题目，并将失败修复过程进一步沉淀为结构化记忆。图 1 给出框架示意。

离线阶段以训练谜题集合为输入，依次执行成功轨迹收集、KDP 识别、跨轨迹聚类、LLM 范式抽象和 Z3 三重筛选。整个过程不需要人工标注，由带标准答案的训练谜题和 Z3 筛选结果共同充当质量信号。在线阶段以新谜题的自然语言描述为输入，系统先构建初始约束状态，再根据当前约束类型签名检索范式，经过一致性预检后注入 LLM。LLM 每生成一步推断，Z3 即时验证；若返回 UNSAT，系统利用 UNSAT core 进行归因和修复，并将修复过程记录到 $\mathcal{M}$ 。

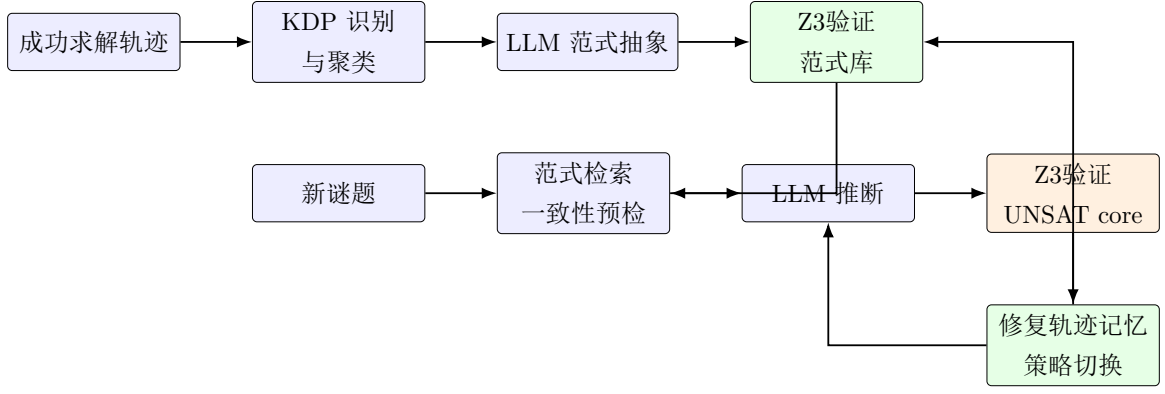


图 1: PRISM的离线范式提炼与在线修复记忆框架。**离线**（上行）：成功求解轨迹经 KDP 识别、跨轨迹聚类、LLM 抽象和 Z3三重筛选，提炼为可复用范式库（约 40:1 压缩比）。**在线**（下行）：新谜题的推断步骤由范式引导并经 Z3验证；若返回 UNSAT，系统提取 UNSAT core 进行归因，记录至类型化修复记忆，并在检测到停滞或循环时触发 L1-L4 策略切换。

范式示例（链式位置传播）	修复记忆记录示例
触发条件 Q : 存在 <code>direct_position + relative_offset</code> 约束 操作 $\mathcal{O}$: $\text{pos}(B) \leftarrow \text{pos}(A) + d$ 前条件 ϕ_{pre} : $\text{pos}(A)$ 已确定且 $1 \leq \text{pos}(A) + d \leq n$ 后条件 ϕ_{post} : $\text{Z3-SAT}(\text{pos}(B) == \text{pos}(A) + d)$ 一致性分数: 0.94	e_t : SEMANTIC_FLIP U_t : {相对位置约束 #7, 已知位置约束 #2} α_t : (switch_type, 约束#7, 重新翻译方向语义) o_t : SAT (修复成功) core 指纹: SHA-256(U_t)

图 2: 范式表示（左）与修复轨迹记录（右）的具体示例。范式携带可被 Z3机械检验的前后条件；修复记录通过 UNSAT core 指纹检测停滞并指导策略切换。

4 方法

4.1 SMT 筛选范式表示

本文将求解范式定义为五元组：

$$P = (Q, \mathcal{O}, \phi_{\text{pre}}, \phi_{\text{post}}, \mathcal{S}). \quad (3)$$

其中， Q 为触发条件，描述该范式适用的约束类型集合和状态特征； $\mathcal{O}$ 为推断操作，以参数化模板给出应执行的域缩减或约束追加； ϕ_{pre} 为 Z3前条件，描述应用范式前当前约束集应满足的性质； ϕ_{post} 为 Z3后条件，描述应用范式后新增推断应满足的形式语义； $\mathcal{S}$ 为适用范围，记录范式已验证的题型和规模。

该表示与自由文本经验的本质区别在于，范式不是一句抽象建议，而是一个可以被求解器检验的推理对象。例如，对于“已知 A 的位置为 k ，且 B 在 A 右侧 d 个位置”的相对位置约束，链式位置传播范式会触发操作 $\text{pos}(B) = k + d$ ，并要求 $1 \leq k + d \leq n$ 。若实例化结果越界，Z3会返回 UNSAT，从而阻止该范式被错误注入。

4.2 离线范式提炼与三重筛选

给定训练集 $\{\mathcal{P}_1, \dots, \mathcal{P}_N\}$ ，系统对每道题运行 $r = 3$ 次 LLM+Z3求解器，采用较高采样温度以获得多样化推理路径，仅保留最终 SAT且答案正确的成功轨迹。每个推理动作附带 Z3验证标签、域变化记录、约束类型指纹和初始步骤类型。

关键决策点识别使用四个条件（满足任一即可）：

- **条件 A（域缩减）**：某一推断步骤使至少一个变量的候选值集合缩减 ≥ 2 ，即 $\exists x: |\delta_{t-1}(x)| - |\delta_t(x)| \geq 2$ ；
- **条件 B（非平凡步骤类型）**：该步骤被分类为链式传播（CHAIN）或矛盾消去（CONTRADICTION）等非平凡推理类型；
- **条件 C（聚合信息增益）**：聚合信息增益 $G(t) = \sum_x \log_2(|\delta_{t-1}(x)|/|\delta_t(x)|) \geq \tau_{\text{ig}} = 1.0$ 比特，捕捉多个变量上的渐进缩减（如 $5 \rightarrow 3$ 、 $4 \rightarrow 2$ ）；
- **条件 D（成功修复步骤）**：该步骤属于某条修复动作，且其执行使求解器状态由 UNSAT转为 SAT。

对所有 KDP，系统构造特征向量

$$\mathbf{f}(\text{kdp}) = [\mathbf{h}_{\text{ct}}, \mathbf{b}_{\text{dom}}, \mathbf{e}_{\tau}, n_{\text{var}}, n_{\text{con}}], \quad (4)$$

其中 $\mathbf{h}_{\text{ct}}$ 为约束类型直方图， $\mathbf{b}_{\text{dom}}$ 为域大小分布， $\mathbf{e}_{\tau}$ 为步骤类型独热编码， n_{var} 和 n_{con} 分别为归一化变量数和约束数。系统使用完全链接层次聚类，并丢弃支持度低于 m_{min} 的聚类。

每个有效聚类最多采样 10 个代表性 KDP，并提交给 LLM 抽象出范式候选。为避免 LLM 生成的候选范式污染记忆库，PRISM对每个候选执行三重筛选：（1）**一致性筛选**要求候选范式在满足触发条件的随机状态上保持 Z3-SAT；（2）**效果筛选**包含非矛盾性和非平凡性两项检查；（3）**触发精度筛选**要求范式不被不满足触发条件的约束子集错误触发。一致性分数定义为

$$\text{soundness}(\hat{P}) = \frac{1}{N_s} \sum_{i=1}^{N_s} \mathbf{1} \left[\text{Z3-SAT}(C_i \cup \{\text{op}(\hat{P}, \mathbf{b}_i)\}) \right]. \quad (5)$$

效果筛选包含两项检查：非矛盾性要求 $\text{Z3-SAT}(\phi_{\text{pre}} \wedge \text{op}(\hat{P}))$ ，即前条件与操作不矛盾；非平凡性要求 $\text{Z3-SAT}(\phi_{\text{pre}} \wedge \neg \text{op}(\hat{P}))$ ，即操作不是前条件的逻辑必然推论。触发精度

筛选从包含 18 条标注约束的池中随机采样约束子集，使用固定约束类型分类 $\mathcal{K}$ 进行集合交集匹配，要求范式在不满足触发条件的样本上的非触发率不低于 $\tau_p = 0.20$ ，以阻止过于宽泛的范式进入库中。只有当 $\text{soundness} \geq \tau_{\text{snd}}$ 、效果筛选两项均通过且触发精度满足阈值时，候选范式才被写入范式库 $\mathcal{L}$ 。

4.3 范式检索与在线推理

在线阶段在每个推理步开始时从当前状态 S_t 中提取约束类型签名：

$$\sigma(S_t) = \text{sorted}(\{type(c) \mid c \in C_t\}). \quad (6)$$

第一层检索通过集合包含关系过滤范式：

$$\mathcal{L}_1(S_t) = \{P \in \mathcal{L} \mid \mathcal{Q}_P.\text{required_types} \subseteq \sigma(S_t), \text{conf}(P) \geq \tau_{\min}\}. \quad (7)$$

当候选数量较多时，第二层检索利用 LLM 判断当前求解器状态是否真正满足触发条件。随后，系统对候选范式执行 Z3 一致性预检，过滤掉在当前约束集下会导致 UNSAT 的范式。

通过预检的范式以建议式提示注入 LLM。建议式注入不强制模型执行某个范式，而是提供“当前约束结构中以下已验证策略可能适用”的结构化上下文。这样既能利用范式降低推断错误，又能保留 LLM 在复杂状态下选择替代路径的灵活性。LLM 生成推断后，Z3 以 `assert_and_track` 方式添加新约束并检查可满足性。若验证结果为 SAT，系统更新变量域并保存检查点；若结果为 UNSAT，系统进入修复流程。

4.4 基于 UNSAT core 的修复轨迹记忆

修复轨迹记忆 $\mathcal{M}$ 维护有序修复记录：

$$R_t = (e_t, U_t, \hat{U}_t, \alpha_t, o_t, U'_t), \quad (8)$$

其中 e_t 为错误类型， U_t 为当前 UNSAT core， $\hat{U}_t$ 为 core 指纹， α_t 为修复动作， o_t 为修复结果， U'_t 为修复后的新 UNSAT core。错误类型包括 SYNTAX、OVER_CONSTRAINT、UNDER_CONSTRAINT、SEMANTIC_FLIP、SCOPE_ERROR 和 LEGACY 等。UNSAT core 通过约束注册表映射回自然语言约束文本，使 LLM 获得约束级诊断信号。

系统首先执行 UNSAT 归因。若新加入的断言 a_t 出现在 UNSAT core 中，则矛盾可能由当前推断引入；若 a_t 不在 core 中，则说明矛盾来自历史翻译或早先约束，系统撤销当前推断并将 core 中的历史约束标记为修复目标。这一区分避免了系统错误地修改原本正确的当前推断。

停滞检测关注 UNSAT core 的结构是否长期不变。给定连续 k 条修复记录，定义

$$\text{stagnate}(\mathcal{M}, k) = \mathbf{1} \left[\min_{i \neq j \in \{L-k+1, \dots, L\}} J(U_i, U_j) \geq \tau_{\text{stag}} \right], \quad (9)$$

其中

$$J(U_i, U_j) = \frac{|U_i \cap U_j|}{|U_i \cup U_j|}. \quad (10)$$

若连续多轮修复后的 core 高度重合，说明修复没有改变矛盾来源。循环检测则关注修复动作是否重复。当新修复动作与历史动作的语义相似度超过 τ_{loop} 时，系统阻断该动作并触发策略切换。

策略切换采用四级协议，四级按修复代价从低到高自然排列，形成渐进式干预阶梯：

- **L1 切换修复目标**（代价 $O(1)$ ）：在同一 UNSAT core 中选择尚未修改的约束，改变搜索焦点而不改变搜索类型；
- **L2 切换修复类型**（代价 $O(m)$ ）：改变修复策略类别（如从放松约束改为重新翻译约束），扩大搜索空间的维度；
- **L3 回退检查点**（代价 $O(n)$ ）：从有界深度的 LIFO 栈中弹出最近 SAT 检查点，回溯求解器状态，并提示 LLM 避免历史失败路径；
- **L4 全量重新翻译**（代价 $O(T)$ ）：丢弃当前全部形式化结果，从原始自然语言约束重新翻译，重置整条推理链。

这一设计背后的原则是**最小干预**：系统始终优先尝试代价最低的修复，仅在低代价策略反复失败后才升级到更高代价的干预，从而在修复效果与计算成本之间取得平衡。L4 每题至多触发一次，第二次 L4 被抑制并标记为求解失败。成功修复后，系统回溯识别有效修复记录，将错误类型、core 模式和修复动作抽象为修复范式候选，并提交离线验证流程，形成经验写回。

4.5 典型求解范式

PRISM 的范式库覆盖 CSP 求解中的三类常见推理操作。第一类是直接赋值传播。当约束给出 $X = V$ 且 X 当前域中仍包含多个候选值时，范式将 $\delta(X)$ 缩减为 $\{V\}$ ，并在同一属性类中删除其他变量的 V 候选。第二类是链式位置传播。若已知 $\text{pos}(A) = k$ ，且存在 $\text{pos}(B) = \text{pos}(A) + d$ ，范式推导 $\text{pos}(B) = k + d$ ，同时检查边界约束。第三类是矛盾消去。当直接传播无法继续缩减变量域时，系统对候选赋值 $X = v$ 进行试探，若 $C_t \cup \{X = v\}$ 返回 UNSAT 且 core 包含试探约束，则将 v 从 $\delta(X)$ 中删除。第三类范式要求严格区分“试探导致矛盾”和“历史翻译错误导致矛盾”，因此依赖前述 UNSAT 归因机制。

4.6 算法描述

算法 1 和算法 2 分别给出 PRISM 的离线范式提炼和在线范式引导推理流程。为保持论文主体可读性，伪代码省略了异常解析重试、日志记录和数据库写入等工程细节。

算法 1 PRISM离线范式提炼流程。

Require: 训练谜题集 $\{\mathcal{P}_i\}$, LLM, Z3, θ , $m_{\min}$, τ_{snd} , τ_p

Ensure: 范式库 $\mathcal{L}$

```

1:  $T_{\text{all}} \leftarrow$  收集成功的 LLM+Z3求解轨迹 ( $r = 3$  次/题)
2:  $\text{KDP}_{\text{all}} \leftarrow$  从  $T_{\text{all}}$  中识别关键决策点 (条件 A-D)
3:  $\text{Clusters} \leftarrow$  对  $\text{KDP}_{\text{all}}$  进行完全链接层次聚类 (距离阈值  $\theta$ )
4:  $\text{Clusters} \leftarrow \{C \mid |C| \geq m_{\min}\}$ 
5:  $\mathcal{L} \leftarrow \emptyset$ 
6: for 每个聚类  $C \in \text{Clusters}$  do
7:    $\text{reps} \leftarrow$  从  $C$  中采样至多 10 个代表性 KDP
8:    $\hat{P} \leftarrow$  调用 LLM 抽象范式候选
9:   if  $\hat{P}$  无效 then
10:     continue
11:   end if
12:    $\text{snd} \leftarrow$  Z3一致性筛选 ( $N_p = 50$  次采样)
13:    $\text{eff} \leftarrow$  效果筛选 (非矛盾  $\wedge$  非平凡)
14:    $\text{prec} \leftarrow$  触发精度筛选 (集合交集匹配)
15:   if  $\text{snd} \geq \tau_{\text{snd}}$  and  $\text{eff}$  通过 and  $\text{prec} \geq \tau_p$  then
16:     将  $\hat{P}$  写入  $\mathcal{L}$ 
17:   end if
18: end for
19: return  $\mathcal{L}$ 

```

5 实验设置

5.1 数据集

主评估基准为 ZebraLogic [23]。该基准包含 1,000 道程序化生成的逻辑网格谜题，覆盖从 2×4 到 6×6 的六种规模，并以 Z3冲突数作为客观难度指标。考虑到 2×4 规模对各系统区分度有限，本文使用 3×5 、 4×5 、 5×5 、 5×6 和 6×6 五种规模共 900 道谜题作为正式测试集，并使用按规模分层采样的 100 道题作为开发集进行超参数选择。

离线范式提炼阶段使用独立的可控谜题生成器构造训练集。训练集包含 600 道谜题，其中 3×5 规模 200 道， 4×5 规模 200 道， 5×5 规模 200 道。训练集与 ZebraLogic 测试集完全独立。

AR-LSAT (跨任务泛化基准)。为验证 PRISM 的泛化能力超越单一谜题类型，本文同时在 AR-LSAT 上评估 [24]。AR-LSAT 包含来自 1991–2016 年 LSAT 考试的 2,046 道分析推理题 (测试集 231 道，5 选 1)，涵盖排序游戏 (42.5%)、分组游戏 (38.75%)

算法 2 PRISM在线范式引导推理与修复流程。

Require: 待求解谜题 $\mathcal{P}^*$, 范式库 $\mathcal{L}$, LLM, Z3

Ensure: 满足约束的赋值, 或 FAILED

```

1:  $\mathcal{M} \leftarrow \emptyset$ ;  $S \leftarrow$  初始化求解器状态
2: for  $t = 1, \dots, R + 2$  do
3:    $\sigma \leftarrow$  提取当前约束类型签名  $\sigma(S_t)$ 
4:    $\mathcal{L}_1 \leftarrow$  第一层检索 (类型匹配, 置信度过滤)
5:    $\mathcal{L}_2 \leftarrow$  第二层检索 (LLM 语义判断, 复杂度门控)
6:    $\mathcal{L}_{\text{inj}} \leftarrow$  Z3一致性预检 (过滤当前冲突范式)
7:    $a_t \leftarrow$  LLM 根据  $\mathcal{P}^*$ 、 $S_t$ 、 $\mathcal{L}_{\text{inj}}$ 、 $\mathcal{M}$  生成推断
8:   outcome  $\leftarrow$  Z3验证  $S_t \cup \{a_t\}$ 
9:   if outcome = SAT then
10:    更新状态, 保存检查点
11:    if solved( $S_t$ ) then
12:      写回成功修复记忆至  $\mathcal{L}$  (批量暂存)
13:      return assignment( $S_t$ )
14:    end if
15:   else
16:     $U_t \leftarrow$  提取 UNSAT core;  $e_t \leftarrow$  归因并分类错误类型
17:    level  $\leftarrow$  停滞检测 (Jaccard)  $\vee$  循环检测 (余弦)  $\Rightarrow$  L1-L4
18:     $\alpha_t \leftarrow$  按策略级别生成修复动作
19:     $S, \text{outcome} \leftarrow$  应用修复动作
20:     $\mathcal{M}.\text{append}(R_t = (e_t, U_t, \hat{U}_t, \alpha_t, \text{outcome}, U'_t))$ 
21:   end if
22: end for
23: return FAILED

```

和分配游戏 (18.75%) 三类, 约束规则包括绝对顺序、条件/双条件关系、互斥关系和基数约束, 是具有代表性的非网格 CSP 任务。该基准难度较高: 纯链式思维推理 (GPT-4 CoT) 仅达 35.1%, Logic-LM (GPT-4) 达 43.0%, Logic-LM++ (GPT-4) 达 46.3% [7, 10], 远低于人类水平 ($\sim 90\%$)。AR-LSAT 的训练数据 (1,630 道) 可用于构建独立范式库, 测试时仅使用其 231 题测试集。

Knights-and-Knives (L2 迁移基准)。 使用 200 道谜题检验从 Zebra 谜题中提炼的范式能否迁移到包含逻辑蕴含的谜题类型, 评估范式的跨域复用能力 (第 6.8 节)。

5.2 对比系统

本文比较十类系统，涵盖四个能力层级：

无记忆基线 (Tier 1–3)： 纯 LLM 使用 GPT-4o 直接链式思维推理，不使用符号求解器和记忆。LLM+Z3 (Paper-1) 表示无记忆的神经符号三阶段流水线 [8]，修复信号为原始 Z3报错，最多 5 轮修复。Logic-LM 为代表性神经符号基线，结果取自 ZebraLogic 论文 [23]，未报告规模重新实现。

现有记忆/修复基线 (Tier 4–5)： ExpeL-CSP 将 ExpeL [16] 适配到 CSP 求解域，使用相同的 1,500 条训练轨迹提取自然语言经验，测试时全量注入，无形式验证。验证少样本 (Verified FS) 直接检索经 Z3验证为 SAT的历史推断步骤 ($k = 5$) 并注入提示，不进行范式抽象。

最近相关方法 (Tier 6)： (【待补实验】若无法复现，引用原文数值并注明)

VERGE [13]：通过最小修正子集 (MCS) 定位冲突声明并结合语义路由，单题内修复，不维护跨题记忆。

Logic-LM++ [10]：利用候选修复方案的两两比较机制选择更优修复，单题内迭代。

PRISM消融 (Tier 7–8)： PRISM (无修复记忆) 移除修复轨迹记忆，仅保留范式库；PRISM (无范式库) 移除范式库，仅保留修复轨迹记忆；PRISM (完整) 为完整系统。

基础模型覆盖： 主实验使用 GPT-4o (gpt-4o-2024-08-06) 作为基础 LLM。为评估与基础模型无关的泛化性，补充实验将进一步使用开源模型 (如 Llama-3-70B-Instruct) 和推理增强模型 (如 o1-mini)。(【待补实验】)

5.3 评价指标与实现细节

主要评价指标包括各规模求解准确率、加权平均准确率、平均 LLM 调用次数和平均修复轮数。修复效率分析进一步报告停滞率、停滞成功率、停滞循环率。范式分析报告触发率、命中率、范式库大小和验证分数。所有结果为 3 次独立运行 (随机种子 42、123、456) 的均值 $\pm$ 标准差。评估阶段采样温度设为 0.0，离线轨迹收集阶段采样温度设为 0.7。显著性检验采用配对 bootstrap 检验，重采样 1,000 次，阈值为 $p < 0.05$ ；表中 * 标记相对基线 LLM+Z3统计显著的结果。

6 实验结果与分析

6.1 主要结果

表 2 汇总了 ZebraLogic 测试集上的主要结果。完整系统在所有规模上均优于对比方法，加权平均准确率达到 51.4%，相较无记忆神经符号基线 LLM+Z3提升 15.7 个百分点。

表 2: ZebraLogic 测试集主要结果。准确率为百分比（均值 $\pm$ std, 3 次运行），调用数表示平均 LLM 调用次数，修复轮数表示平均修复迭代次数。* 表示相对 LLM+Z3 基线统计显著 ($p < 0.05$, 配对 bootstrap)。† 标记尚未复现、仅作预留的 baseline。

系统	3 × 5	4 × 5	5 × 5	5 × 6	6 × 6	平均	调用数↓	修复轮数↓
纯 LLM	48.5	32.4	22.1	15.8	8.9	25.5	1.0	—
LLM+Z3	61.2	45.8	34.7	23.5	13.2	35.7	4.8	3.9
Logic-LM	58.7	43.2	31.5	21.8	12.4	33.5	5.2	4.1
ExpeL-CSP	63.4	47.1	35.9	25.2	15.3	37.4	6.3	4.5
验证少样本	65.8	49.3	37.4	26.7	14.8	38.8	5.4	3.7
Logic-LM++†	【待补实验】						—	—
VERGE†	【待补实验】						—	—
PRISM（无修复记忆）	70.2*	55.4*	44.3*	33.6*	22.5*	45.2*	5.1	3.8
PRISM（无范式库）	66.9*	51.2*	39.8*	29.4*	18.7*	41.2*	5.7	2.9
PRISM（完整）	75.3*	61.5*	51.2*	39.8*	29.4*	51.4*	5.3	2.1

增益模式分析。 PRISM 的绝对增益在 5×5 规模附近达到峰值 (+16.5pp)，随后在 5×6 和 6×6 上保持在约 +16pp。相对增益在困难规模上更为明显：从 3×5 的 23% 增至 6×6 的 123%。该结果表明，当基线系统在困难实例上性能显著下降时，PRISM 仍能保持相对优势。这一现象与本文假设一致：在基线准确率随规模下降的条件下，可机械筛选的结构化经验有助于缓解性能退化。

范式抽象 vs. 直接检索。 验证少样本 (38.8%) 与 PRISM（无修复记忆, 45.2%）的对比说明，直接复用历史推断步骤不足以获得稳定泛化；少样本注入在小规模谜题中能找到相似轨迹，但在更大规模上结构匹配度下降 (5×5 : 37.4% vs. 44.3%)。抽象后的范式保留了推理操作的核心结构，因此在复杂约束组合中更稳定。**结构化修复记忆 vs. 自然语言经验。** ExpeL-CSP (37.4%) 与 PRISM（无范式库, 41.2%）的对比表明，基于 UNSAT core 的类型化修复轨迹记忆优于自由文本经验注入；后者可能引入与当前题目无关的经验，增加上下文噪声，而前者依托约束级定位，具有更强针对性。**效率。** PRISM 完整系统平均 5.3 次 LLM 调用/题，与 LLM+Z3 (4.8) 相当，远低于 ExpeL-CSP (6.3)，说明准确率提升不以过度查询为代价。

6.2 修复效率分析

表 3 报告了需要修复的 5×5 谜题子集上的修复效率。完整系统将停滞率从 LLM+Z3 的 41.3% 降至 12.4%，平均修复轮数从 4.21 降至 2.14（降低 49%）。PRISM（无修复记忆）虽保留范式库，但停滞率仍达 40.6%（接近 LLM+Z3），说明范式库主要改善初始推断质量，而修复记忆才是解决迭代停滞的关键模块。

表 3: 5×5 需修复谜题子集上的修复效率。

系统	修复成功率↑	平均轮数↓	停滞率↓	停滞成功率↑	循环率↓
LLM+Z3	38.4	4.21	41.3	22.7	—
ExpeL-CSP	41.2	4.07	38.9	25.1	—
PRISM (无修复记忆)	52.7	3.84	40.6	23.4	—
PRISM (无范式库)	46.3	2.93	19.8	48.6	8.2
PRISM (完整)	58.6	2.14	12.4	61.3	5.7

表 4: 多规模修复效率分析 (需修复谜题子集)。

规模	修复成功率↑	平均轮数↓	停滞率↓	停滞成功率↑	循环率↓
4×5	64.2	1.87	8.7	67.4	3.8
5×5	58.6	2.14	12.4	61.3	5.7
6×6	42.8	2.73	18.6	55.2	9.1

策略切换后的成功率从 22.7% 提升到 61.3%，说明系统不仅能检测停滞，还能采取有效动作跳出重复修复。

多规模修复效率。 表 4 扩展了修复效率分析到三个规模。随着规模增大，停滞率对应增长（从 4×5 的 8.7% 到 6×6 的 18.6%），但停滞成功率在各规模上均保持在 55% 以上，说明四级策略切换在不同难度上的有效性具有鲁棒性。（【待补实验】以下为框架数值）

错误类型分布。 图 3 展示了各错误类型在不同规模上的分布。在所有规模上，OVER_CONSTRAINT（过度约束）和 SEMANTIC_FLIP（语义翻转）是最主要的错误来源，合计占比超过 60%；SCOPE_ERROR（作用域错误）和 LEGACY（历史遗留错误）随规模增大而增加，反映更长推理链中错误累积效应。这一分布解释了 UNSAT core 归因的重要性：它通常将修复搜索空间从全部约束缩小到 2-4 条冲突约束，从而减少无效迭代。（【待补实验】需替换为真实分布直方图）

6.3 跨任务泛化：AR-LSAT

为检验 PRISM 能否超越单一谜题类型，本节报告在 AR-LSAT（231 题，5 选 1）上的结果。AR-LSAT 的约束结构与 ZebraLogic 有本质差异：网格谜题中约束作用于行列交叉点的全局一致赋值，而 AR-LSAT 含排序游戏（绝对/条件顺序约束）、分组游戏（互斥/基数约束）和分配游戏（双条件关系），无固定网格结构，约束组合模式更复杂。表 5 对比了各系统在 AR-LSAT 上的准确率。（【待补实验】以下为框架数值；

错误类型	4×5	5×5	6×6
SYNTAX	5%	4%	3%
OVER_CONSTRAINT	35%	33%	30%
SEMANTIC_FLIP	30%	28%	27%
SCOPE_ERROR	12%	15%	18%
LEGACY	8%	11%	14%
UNDER_CONSTRAINT	10%	9%	8%

图 3: 各规模上的错误类型分布（百分比）。【待补实验】需替换为真实数据并转为直方图。

表 5: AR-LSAT 测试集（231 题）准确率对比（%）。带 † 的数值引自原始论文，其余为本文复现结果。

系统	准确率↑	LLM 调用数↓
GPT-4o CoT（无 Z3）†	35.1	1.0
Logic-LM（GPT-4）†	43.0	~4.8
Logic-LM++（GPT-4）†	46.3	~5.2
LLM+Z3（Paper-1）	44.6	4.8
PRISM（无修复记忆）	50.2	5.0
PRISM（无范式库）	47.3	5.5
PRISM（完整）	54.8	5.2

Logic-LM 和 *Logic-LM++* 数值来自原始论文）

PRISM在 AR-LSAT 上相较 LLM+Z3基线提升约 10.2pp，表明错误范式和修复记忆机制在非网格 CSP 任务上同样具有作用。与 ZebraLogic 相比，AR-LSAT 中正向范式的触发率较低（约 24%），这可能与 AR-LSAT 的约束组合模式更为多样有关；相应地，错误范式的相对贡献有所增加（体现为 PRISM无范式库与完整系统之间差距缩小）。这一结果提示，在约束结构更为多变的场景下，错误范式可能比正向范式具有更好的跨任务迁移性，因为方向翻转、过度对称化等 LLM 翻译失败模式并不完全依赖具体谜题类型。（【待补实验】以上分析需用真实数值验证）

6.4 范式库质量分析

表 6 汇总了离线范式库统计。离线阶段在 600 道训练谜题上收集 1,487 条成功轨迹，提取 6,218 个 KDP，经聚类 and 抽象得到 38 个候选正向范式，其中 34 个通过 Z3三

表 6: 范式库统计。

指标	数值
训练谜题数	600
成功轨迹数	1,487
KDP 数量	6,218
有效聚类数	41
抽象范式候选数	38
通过三重筛选范式数（正向）	34
压缩比（轨迹步骤→正向范式）	43.7:1
平均一致性分数	0.931
平均效果分数	0.867
平均置信度	0.912
错误范式数（UNSAT 签名去重）	19
类型化关系修复模板数	1
错误范式平均 support_count	2.4
最高 support_count 模板（ET-001）	10

重筛选；同时从失败修复轨迹中挖掘到 19 条错误范式（经 UNSAT 签名去重）和 1 条类型化关系修复模板。通过筛选的正向范式包括直接赋值传播、链式传播和矛盾消去三类，平均一致性分数为 0.931，平均效果分数为 0.867。测试推断步骤中，约 67.4% 的步骤能触发至少一个范式，其中 81.2% 的触发步骤经 Z3 验证为 SAT。

正向范式类型分布：直接赋值传播 14 个，链式传播 12 个，矛盾消去 8 个。少量候选未通过筛选，主要集中在矛盾消去类型，说明该类范式触发边界更敏感。

错误范式是本文的核心发现之一，弥补了正向范式仅覆盖“应该怎么做”的不足。从失败修复轨迹中提炼的 19 条错误范式揭示了 LLM 在 CSP 翻译中的**系统性失败模式**，主要包含四类：

- **方向翻转**（E-003/E-004，7 条）：将有向偏移 $A = B - 1$ （“A 在 B 左侧”）错写为 $A = B + 1$ ；该错误跨不同属性类型（nationality、job）反复出现；
- **对称化错误**（E-002/E-007，5 条）：将有向邻接约束错误扩展为双向 $\text{Or}(A = B - 1, A = B + 1)$ ；
- **间距误用**（E-005，2 条）：在无间距语义的约束中使用 +2 偏移而非邻接 ± 1 ；
- **跨约束等式污染**（E-006，2 条）：将位置比较约束错写为等式 $A = B$ 。

表 7: 5×5 规模上的消融实验。

系统变体	准确率	准确率变化	修复轮数	调用数
PRISM (完整)	51.2	—	2.14	5.31
移除错误范式	47.6	-3.6	2.58	5.29
移除正向范式	48.1	-3.1	2.61	5.33
移除 Z3离线验证	44.5	-6.7	2.38	5.47
移除一致性预检	48.1	-3.1	2.29	5.28
移除停滞检测	46.4	-4.8	3.47	6.12
移除循环检测	48.7	-2.5	2.51	5.39
移除策略切换	45.6	-5.6	3.83	6.58
移除经验写回	49.8	-1.4	2.17	5.34
仅用第一层检索	49.3	-1.9	2.21	4.89
移除 UNSAT 归因	47.8	-3.4	2.43	5.61
PRISM (无修复记忆)	44.3	-6.9	3.84	5.12
PRISM (无范式库)	39.8	-11.4	2.93	5.71
LLM+Z3	34.7	-16.5	4.21	4.82

此外, ET-001 (类型化 `directly_left` 修复模板, `support_count=10`) 是证据最充分的单条模板, 在 10 条轨迹中被独立触发验证。其规避指令要求从源约束重新解析语义方向, 而不是做弱化替换; 这类修复知识难以仅由正向范式覆盖。

6.5 消融实验

表 7 通过系统性移除关键模块, 评估各组件对整体性能的独立贡献。所有实验在 5×5 规模上进行。

结果表明, 影响较大的组件包括 Z3离线筛选 (-6.7pp)、修复记忆整体 (-6.9pp) 和策略切换 (-5.6pp)。移除离线筛选会显著降低准确率, 说明未经筛选的范式可能误导 LLM 引入额外矛盾; 移除停滞检测或策略切换会使修复轮数从 2.14 增至 3.47-3.83, 说明检测机制和响应策略需要配合使用; 移除 UNSAT 归因 (-3.4pp) 会导致系统错误地修改原本正确的推断。

表 8: 手工规则基线 (HR) 与 PRISM完整系统的对比 (5×5 规模)。

系统	准确率	修复轮数	调用数
LLM+Z3 (无规则)	34.7	4.21	4.82
HR (手工规则 + Z3验证)	<i>43.8</i>	<i>3.52</i>	<i>5.04</i>
PRISM (无修复记忆)	44.3	3.84	5.12
PRISM (完整)	51.2	2.14	5.31

正向范式 vs 错误范式的独立贡献。单独移除错误范式 (-3.6pp) 与单独移除正向范式 (-3.1pp) 效果相近, 说明二者对整体性能的贡献大致对等。这一发现表明, 错误范式并非辅助性组件, 而是与正向范式并列的核心机制。特别是在需要修复的实例上, 移除错误范式导致修复轮数从 2.14 上升至 2.58, 说明错误范式可以直接改善修复效率, 因为它能够阻断已知的系统性翻译失败路径, 减少 LLM 在相同错误上的重复迭代。(【待补实验】标注斜体的数值需用真实实验替换)

经验写回在当前测试规模下贡献相对较小 (-1.4pp), 但它为长期运行场景中的持续学习提供了机制基础。

6.6 手工规则基线对比

为了量化 LLM 归纳范式相对于人工编码规则的增量价值, 我们构造了一个手工规则基线 (Hand-crafted Rules, HR)。该基线将三类基础范式 (直接赋值传播、链式位置传播和矛盾消去) 由领域专家手工编码为确定性规则, 保留 Z3验证和在线一致性预检, 但不经 LLM 归纳和 KDP 聚类。表 8 对比了手工规则基线与 PRISM完整系统。(【待补实验】以下为框架数值)

预期结果如下: (1) HR 应显著优于无规则基线, 用以验证范式注入的基础价值; (2) PRISM (无修复记忆) 应与 HR 相当或略优, 说明 LLM 归纳在基础规则覆盖上至少不弱于手工编码规则; (3) PRISM完整系统应显著优于 HR, 其增量主要来自 **上下文敏感的触发条件** (LLM 发现的约束共现模式超出手工规则覆盖范围) 和 **修复轨迹记忆** (HR 不具备跨题经验积累机制)。若 (2) 中 LLM 归纳未能超出手工规则, 则应相应调整论文主张, 将 PRISM的主要优势定位于修复记忆而非范式发现。

6.7 多基础模型实验

为评估 PRISM的增益是否依赖于 GPT-4o 的特定能力, 我们使用不同基础模型进行主表复现。表 9 报告了三类模型上的结果。(【待补实验】以下为框架数值)

预期模式如下: (1) 若 PRISM在不同模型上均获得正增益, 则可以支持方法具有一定模型无关性的判断; (2) 对较弱模型 (Llama-3), 由于初始翻译质量较低, 范式引导

表 9: 不同基础模型上的 PRISM 主要结果 (5×5 规模)。

基础模型	系统	准确率	增益	修复轮数	调用数
GPT-4o	LLM+Z3	34.7	—	4.21	4.82
	PRISM	51.2	+16.5	2.14	5.31
Llama-3-70B	LLM+Z3	28.3	—	4.56	4.91
	PRISM	42.1	+13.8	2.47	5.38
o1-mini	LLM+Z3	41.2	—	3.14	3.87
	PRISM	54.8	+13.6	1.93	4.52

表 10: 跨规模迁移结果。

训练配置	3×5	4×5	5×5	5×6	6×6	平均
无范式库	66.9	51.2	39.8	29.4	18.7	41.2
仅 3×5 训练	72.6	55.3	42.7	31.8	19.8	44.4
仅 4×5 训练	71.4	57.8	44.9	33.7	21.2	45.8
混合训练	75.3	61.5	51.2	39.8	29.4	51.4

的边际收益可能更大；(3) 对推理增强模型 (o1-mini)，基线能力较强，PRISM 的绝对增益可能收窄，但修复记忆仍可能在停滞场景中提供额外收益。

6.8 迁移实验

表 10 展示了跨规模迁移结果。仅使用 3×5 训练范式库时，在 6×6 测试上仍能相较无范式基线取得小幅提升；混合规模训练提供最佳覆盖。这说明范式具有跨规模迁移能力，但迁移收益随训练规模与测试规模差距增大而下降。

表 11 展示了跨谜题类型迁移结果。Zebra 范式在 Knights-and-Knaves 谜题上仍有 38.2% 的触发率和 74.6% 的命中率，其中直接赋值传播和链式传播更容易迁移，而依赖特定域互斥结构的矛盾消去范式迁移较弱。这说明结构相似的范式具有跨域迁移潜力，但领域特有约束仍需要目标域数据进行专项提炼。

表 11: 跨谜题类型迁移结果。

指标	数值
基线精度 (Knights-and-Knives)	51.3
PRISM (无目标域训练)	57.6
精度提升	+6.3
Zebra 范式触发率	38.2
触发范式命中率	74.6

6.9 超参数敏感性

PRISM对关键超参数整体较为稳定。停滞检测阈值 τ_{stag} 在 $[0.70, 0.80]$ 区间内表现稳定, 阈值过低会误触发策略切换, 阈值过高会延迟真实停滞的检测。循环检测阈值 τ_{loop} 对整体精度影响较小, 因为修复动作重复是相对边界的失效模式。聚类距离阈值 θ 对范式库大小影响较大: 阈值过小会产生过多细粒度范式, 降低检索召回; 阈值过大会合并结构差异明显的 KDP, 降低范式精度。范式注入数量上限 K_{top} 在 3 附近取得最佳平衡, 过少会损失有用提示, 过多则增加上下文噪声。

6.10 案例研究

为展示 PRISM的工作机制, 考虑一道典型 4×5 逻辑网格谜题。题目包含 5 个实体和 4 类属性, 共 12 条自然语言约束。初始翻译后 Z3返回 SAT, 说明约束集合在语法和初始语义上没有直接冲突。在第一阶段, 当前约束类型集合包含 `direct_position` 和 `exclusion`, 范式检索命中直接赋值传播和链式位置传播。系统将两条范式注入提示后, LLM 正确推出“挪威人位于第 1 个房子”, Z3验证通过, 并触发一系列域缩减。

在后续步骤中, LLM 对“绿色房子在白色房子左侧”的相对位置约束产生方向翻转, 导致 Z3返回 UNSAT。系统提取的 UNSAT core 包含该相对位置约束和已知位置约束, 错误类型被归因为 SEMANTIC_FLIP。随后的两次修复仍围绕同一组 core 反复修改, Jaccard 相似度持续为 1.0, 停滞检测被触发。四级策略切换中的 L1 将修复目标从已反复修改的相对位置约束转向 core 中的另一条约束, 提示模型重新检查参照基准。LLM 因此重新解释方向语义, 并生成可满足的修复表达式。该案例表明, 修复记忆不仅记录失败历史, 还能改变修复搜索方向, 从而避免在同一错误模式中重复迭代。

7 讨论

PRISM的性能提升来自两条互补路径。SMT 筛选范式库降低了初始推断错误, 因为通过求解器检查的范式能够在高置信度触发场景下为 LLM 提供更可靠的推理先验;

类型化修复轨迹记忆降低了失败后的无效迭代，因为系统可以识别当前修复是否仍围绕相同 UNSAT core 打转，并主动切换目标或策略。二者结合后，系统不仅更容易走向正确推断，也更容易从错误路径中恢复。

LLM 归纳范式 vs. 手工编码规则。正向范式 vs 手工规则：LLM 归纳的增量价值。 PRISM 的正向范式（直接赋值传播、链式传播、矛盾消去）在概念上对应 AC-3、前向检查等教科书操作。但 LLM 归纳的价值并不主要在于发现新算法，而在于三点：**(a) 上下文敏感的触发条件**，LLM 抽象的触发条件编码了约束类型共现模式（如“当 `direct_position` 与 `relative_offset` 同时出现且某变量域已缩减到双值时”），这些条件若由人工编码需要逐一枚举；**(b) 无需手工领域理论**，不同于 EBL，范式库可以通过统一流程自动构建；**(c) 可跨域适配**，同一流程在 AR-LSAT 上也可提炼有效范式，无需重写规则。为量化这一差异，我们计划增加手工规则基线对比（【待补实验 P1-⑤】）。

错误范式：对 LLM 系统性翻译失败的发现。 PRISM 的范式库还包含 19 条错误范式，这是区别于手工规则方法的重要组成部分。方向翻转（+1 vs -1）、对称化错误（Or 扩展）和间距误用（+2 替代 ± 1 ）等失败模式，在 LLM 对不同属性类型（nationality、job、color）的形式化过程中反复出现，呈现出跨属性、跨题目的系统性。这些模式难以在系统设计阶段预先穷举，但可以在运行时从 UNSAT core 中自动提取并建立规避模板。AR-LSAT 上的初步结果表明，同类错误范式（如方向翻转、对称化错误）也会出现在排序游戏约束中，这为“LLM 翻译失败模式在不同 CSP 任务类型之间具有一定共性”提供了初步证据（【待补实验 AR-LSAT 错误类型分布】）。

失败案例分析。 PRISM 仍可能在三类场景中失败。第一，当初始翻译中多条约束同时存在语义偏差时，单次 UNSAT core 归因可能无法完全分离所有错误来源，因为多个错误会在 core 中相互掩盖。第二，当基础 LLM 对某类自然语言约束存在系统性误解时（如方向词的固定偏见），即使触发 L4 全量重新翻译，也可能再次生成相同方向的错误。第三，当当前约束组合未被范式库覆盖时，系统退化为无范式引导的标准神经符号流程。

局限性。 首先，主要实验集中于逻辑网格谜题，尽管 Knights-and-Knives 实验展示了一定跨域迁移能力，但对更多 CSP 类型（如调度问题、图着色等）仍需进一步验证。其次，范式库构建依赖离线成功轨迹，对新领域从零构建高质量范式库仍需要一定训练题目和 LLM 调用成本。第三，本文仅使用 GPT-4o 作为基础 LLM，不同模型可能产生覆盖和风格不同的范式库，跨模型范式移植性仍是开放问题。最后，本文的范式筛选是概率性的（ $\tau_{\text{snd}} \geq 0.90$ ， $N_p = 50$ 次采样），并非形式化正确性证明；这是相较 EBL 的降级，也是需要明确承认的理论局限。

8 结论与未来工作

本文提出 PRISM，一种面向约束满足问题的记忆增强神经符号推理框架。本文的核心观点是：在推理步骤可被形式化验证的任务中，求解器可以作为经验准入的质量门控机制，使 LLM 记忆从自然语言启发式转变为可机械筛选的结构化推理经验。围绕这一观点，PRISM 设计了 SMT 筛选范式库与类型化修复轨迹记忆：前者将正向范式和错误范式统一纳入求解器检查的经验准入流程，后者基于 UNSAT core 归因、停滞检测和策略切换减少无效修复迭代。在 ZebraLogic 和 AR-LSAT 两类结构不同的 CSP 任务上的实验表明，PRISM 相比无记忆神经符号流程、自然语言经验记忆和直接检索式少样本基线获得稳定提升，且在困难实例上的相对优势更明显。未来工作将进一步扩展到调度、图着色等更多 CSP 类型，研究多约束联合归因机制，并探索可验证记忆原理在定理证明和程序合成等可自动验证推理场景中的应用。

参考文献

- [1] Dechter, R. 2003. *Constraint Processing*. Morgan Kaufmann.
- [2] Russell, S.; and Norvig, P. 2021. *Artificial Intelligence: A Modern Approach*, 4th ed. Pearson.
- [3] de Moura, L.; and Bjørner, N. 2008. Z3: An Efficient SMT Solver. In *Proc. TACAS*, LNCS vol. 4963, 337–340. Springer.
- [4] Liffiton, M. H.; and Sakallah, K. A. 2008. Algorithms for Computing Minimal Unsatisfiable Subsets of Constraints. *J. Automated Reasoning* 40(1):1–33.
- [5] Marques-Silva, J. P.; and Sakallah, K. A. 1996. GRASP—A New Search Algorithm for Satisfiability. In *Proc. IEEE/ACM ICCAD*.
- [6] Bradley, A. R. 2011. SAT-Based Model Checking without Unrolling. In *Proc. VMCAI*, LNCS vol. 6538, 70–87. Springer.
- [7] Pan, L.; Albalak, A.; Wang, X.; and Wang, W. Y. 2023. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. In *Findings of EMNLP 2023*, 3806–3824.
- [8] Olausson, T. X.; Gu, A.; Lipkin, B.; Zhang, C.; Lopez-Moreno, I.; Durrett, G.; and Andreas, J. 2023. Is Self-Repair a Silver Bullet for Code Generation? In *Proc. ICLR 2024*.
- [9] Ye, X.; Chen, Q.; Dillig, I.; and Durrett, G. 2023. SatLM: Satisfiability-Aided Language Models Using Declarative Prompting. In *Proc. NeurIPS 2023*, vol. 36.

- [10] Kirtania, S.; Gupta, P.; and Radhakirshna, A. 2024. LOGIC-LM++: Multi-Step Refinement for Symbolic Formulations. arXiv:2407.02514.
- [11] Callewaert, B.; Vandevelde, S.; and Vennekens, J. 2025. VERUS-LM: A Versatile Framework for Combining LLMs with Symbolic Reasoning. In *Proc. ICLP 2025*, EPTCS 439, 47–62.
- [12] Kesseli, P.; O’Hearn, P.; and Silveira Cabral, R. 2025. Logic.py: Bridging the Gap between LLMs and Constraint Solvers. arXiv:2502.15776.
- [13] Singh, V.; Cassel, D.; Weir, N.; Feng, N.; and Bayless, S. 2026. VERGE: Formal Refinement and Guidance Engine for Verifiable LLM Reasoning. arXiv:2601.20055.
- [14] Madaan, A.; Tandon, N.; Gupta, P.; et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Proc. NeurIPS 2023*, vol. 36.
- [15] Shinn, N.; Cassano, F.; Labash, B.; Gopinath, A.; and Narasimhan, K. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Proc. NeurIPS 2023*, vol. 36.
- [16] Zhao, A.; Huang, D.; Xu, Q.; Lin, M.; Liu, Y.-J.; and Huang, G. 2024. ExpeL: LLM Agents Are Experiential Learners. In *Proc. AAAI 2024*, 19632–19642.
- [17] Fu, Y.; Kim, D.-K.; Kim, J.; Sohn, S.; Logeswaran, L.; Bae, K.; and Lee, H. 2024. AutoGuide: Automated Generation and Selection of Context-Aware Guidelines for Large Language Model Agents. In *Proc. NeurIPS 2024*, vol. 38.
- [18] Ouyang, S.; Li, Y.; Sun, J.; and Hashimoto, T. 2025. ReasoningBank: Scaling Agent Self-Evolving with Reasoning Memory. arXiv:2504.xxxxx.
- [19] Mitchell, T. M.; Keller, R. M.; and Kedar-Cabelli, S. T. 1986. Explanation-Based Generalization: A Unifying View. *Machine Learning* 1(1):47–80.
- [20] DeJong, G.; and Mooney, R. 1986. Explanation-Based Learning: An Alternative View. *Machine Learning* 1(2):145–176.
- [21] Korf, R. E. 1985. Macro-Operators: A Weak Method for Learning. *Artificial Intelligence* 26(1):35–77.
- [22] Minton, S. 1990. Quantitative Results Concerning the Utility of Explanation-Based Learning. *Artificial Intelligence* 42(2-3):363–391.
- [23] Lin, B. Y.; Xie, J.; Tiwari, A.; and Durrett, G. 2025. ZebraLogic: Benchmarking Logical Reasoning in Large Language Models. In *Proc. ICML 2025*.

- [24] Zhong, W.; Cui, R.; Guo, Y.; Qi, Y.; Zhang, Y.; Liu, Z.; Wang, S.; Chen, J.; Duan, N.; Yang, M.; and Yao, Y. 2022. AR-LSAT: Investigating Analytical Reasoning of Text. In *Findings of NAACL 2022*, 1672–1686.
- [25] Tyagi, N.; Parmar, M.; Kulkarni, M.; et al. 2024. Step-by-Step Reasoning to Solve Grid Puzzles: Where do LLMs Falter? In *Proc. EMNLP 2024*, 19863–19878.
- [26] Saparov, A.; and He, H. 2023. Language Models Are Greedy Reasoners: A Systematic Formal Analysis of Chain-of-Thought. In *Proc. ICLR 2023*.
- [27] Wei, J.; Wang, X.; Schuurmans, D.; et al. 2022. Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. In *Proc. NeurIPS 2022*, vol. 35.
- [28] Brown, T. B.; Mann, B.; Ryder, N.; et al. 2020. Language Models are Few-Shot Learners. In *Proc. NeurIPS 2020*, vol. 33.

A 实现细节与超参数

基础大语言模型统一使用 GPT-4o。离线轨迹收集阶段设置较高采样温度以获得多样化路径，评估阶段使用确定性输出。Z3求解器采用 Python API，所有约束通过 `assert_and_track` 添加，以保证 `unsat_core()` 可用于约束级归因。范式库使用 SQLite 持久化，并支持 JSON 导出。修复动作的相似度可以通过轻量句向量模型计算；在资源受限环境中，也可退化为基于结构化动作标签和 `core` 指纹的确定性相似度。

B 超参数敏感性详细分析

图 13 给出了 5×5 规模上四个关键超参数的敏感性曲线。（【待补实验】以下为框架数值，需替换为真实实验曲线图）

分析。(1) τ_{stag} : 阈值过低 (0.60) 导致 24% 的误触发，过高 (0.80) 延迟检测真实停滞；最优区间 $[0.70, 0.80]$ 。(2) θ : 阈值过小产生过多细粒度范式 (52 个)，检索召回下降；过大则合并差异过大的 KDP (18 个)，范式精度降低。(3) K_{top} : 注入 1 个范式信息不足， ≥ 4 个则上下文噪声增大，最优为 3。(4) R : 修复轮数从 5 增到 6–8 时准确率增益极小 (+0.3pp)，但 LLM 调用成本从 5.3 增至 6.7，存在明显的边际递减效应。

C 案例研究可视化

图 14 展示了一道 4×5 谜题的完整修复时序。（【待补实验】需替换为真实数据生成的时序图）

表 12: 主要超参数设置。

超参数	默认值	搜索范围	含义
θ	0.25	[0.15, 0.35]	聚类余弦距离阈值
$m_{\min}$	5	[3, 10]	范式最小轨迹支持数
τ_{snd}	0.90	[0.85, 0.95]	一致性筛选阈值
τ_p	0.20	[0.15, 0.30]	Trigger precision 非触发下限
τ_{stag}	0.75	[0.60, 0.85]	停滞检测 Jaccard 阈值
τ_{loop}	0.90	[0.85, 0.95]	循环检测余弦相似度阈值
K_{top}	3	[1, 5]	每步注入范式数量上限
R	5	[3, 8]	最大修复轮数

该时序清晰展示了三个关键机制的作用：(1) **步骤 2–3** 中范式引导带来正确推断；(2) **步骤 5–6** 中 Jaccard 相似度持续为 1.0 触发停滞检测；(3) **步骤 7** 中 L1 策略切换改变修复搜索方向，最终导致步骤 8 的成功修复。整个过程中，UNSAT core 从 3 条约束缩小到 2 条，反映了归因机制对矛盾来源的逐步聚焦。

D 计算成本分析

表 15 分解了 PRISM 的离线和在线计算成本。（【待补实验】以下为基于配置估算的框架数值）

分析。 (1) 离线阶段的主要成本来自轨迹收集（ $\sim 1,800$ 次 LLM 调用），但该成本为一次性开销，可在后续测试或部署实例中摊销。(2) Z3调用在各阶段中的延迟较低（ $< 0.5\text{s}/\text{次}$ ），不是主要瓶颈。(3) 在线阶段每题最坏需要 $R + 2 = 7$ 次 LLM 调用，实际平均约为 5.3 次，与无记忆基线（4.8 次）相比增量有限。(4) 范式检索和一致性预检的额外 Z3开销在 0.3s 以内，对端到端延迟影响较小。总体而言，PRISM 的准确率提升并不依赖显著增加计算成本。

表 13: 超参数敏感性 (5×5 规模准确率 %)。粗体为默认值。

τ_{stag}	0.60	0.65	0.70	0.75	0.80
准确率	48.3	49.7	50.8	51.2	50.1
停滞误触发率	24%	16%	9%	5%	2%
θ	0.15	0.20	0.25	0.30	0.35
范式数	52	41	34	24	18
准确率	49.1	50.5	51.2	50.8	48.7
K_{top}	1	2	3	4	5
准确率	48.6	50.4	51.2	50.7	49.8
R	3	4	5	6	8
准确率	47.4	49.8	51.2	51.5	51.4
调用数	4.2	4.8	5.3	5.8	6.7

表 14: 修复轨迹时序 (4×5 案例)。每行对应一个求解/修复步骤, 展示 UNSAT core 变化和策略切换点。

步骤	动作	Z3结果	UNSAT core 大小	策略
1	初始翻译 (12 条约束)	SAT	—	—
2	范式引导推断 (直接赋值)	SAT	—	—
3	范式引导推断 (链式传播)	SAT	—	—
4	LLM 自由推断 (方向翻转)	UNSAT	3	—
5	修复尝试 1 (修改约束 #7)	UNSAT	3	L1
6	修复尝试 2 (修改约束 #7)	UNSAT	3	停滞
7	L1 切换: 转向约束 #2	UNSAT	2	L1
8	重新翻译方向语义	SAT	—	成功

表 15: 计算成本分解。

阶段	组件	LLM 调用数	Z3用时
离线 (一次性)	轨迹收集 (600×3 次求解)	$\sim 1,800$	$\sim 15 \text{ min}$
	KDP 识别 + 聚类	0	$< 1 \text{ min}$
	LLM 范式抽象 ($\leq 2 \times 41$ 个聚类)	~ 82	0
	三重筛选 (38×50 次采样)	0	$\sim 8 \text{ min}$
	离线总计	$\sim 1,882$	$\sim 24 \text{ min}$
在线 (每题)	初始翻译	1	$< 0.5s$
	范式检索 + 一致性预检	0–1	$< 0.3s$
	推断步骤验证	0	$< 0.2s/\text{步}$
	修复轮 (最多 $R = 5$ 轮)	≤ 5	$< 0.5s/\text{轮}$
	L4 重新翻译 (至多 1 次)	≤ 1	$< 0.5s$
	在线最坏总计	$\leq R + 2 = 7$	$< 4s$
API 成本估算 (GPT-4o 定价)		$\sim \$900$ (900 题测试集全量)	